

IODA-ARCHITEKTUR, TEIL 1

Eine Kritik bisheriger Architekturmodelle

Vom Monolithen über Schichten hin zu IODA (Integration, Operation, Daten, APIs).

Ich kann sie nicht mehr hören: die Anpreisungen des Architekturmodells „**Schichtenmodell**“. In der dotnetpro wie anderswo spukt es immer wieder als klassische und deshalb gute Organisation von Code herum. Mir scheint das inzwischen ein Fall von Cargo-Kult: Irgendwer hat irgendwann seinen Code so strukturiert und damit einen Vorteil erlangt – und nun **folgen** ihm Generationen von Entwicklern **blind**.

Was aber, wenn sich die Welt weitergedreht hat? Was, wenn man da etwas mechanisch tut, ohne wirklich konsequent über die ursprünglichen Beweggründe nachzudenken? Das Ergebnis ist dann immer gleich: Es entsteht Unwohlsein, die **Dinge werden schwierig – doch man weiß nicht**

so recht, warum. Man macht doch **alles richtig**, oder? Eher wohl nicht; vielleicht muss man sich einfach noch mehr **bemühen**. Also die **Anstrengungen verdoppeln**, das Muster einzuhalten. Und so entsteht dann noch **mehr Unwohlsein**.

„Been there, done that, got the t-shirt“, kann ich dazu sagen. Einst war auch ich ein Anhänger des Schichtenmodells und seiner mustergültigen Geschwister. Doch irgendwann habe ich für mich realisiert: Der Schmerz ist größer als der Nutzen. Ich muss die Muster nicht besser anwenden, sondern **andere Wege** suchen, meine **Software zu strukturieren**.

Worauf ich dann gestoßen bin, davon möchte ich Ihnen hier berichten. Es ist eine Geschichte der **Erleichterung**. Softwareentwicklung macht mir wieder **Spaß**. Ich kann mich viel mehr auf die **Lösung** der Probleme **konzentrieren**, weil die **Struktur** mich **nicht mehr in ein hinderliches Korsett zwingt**.

Aber der Reihe nach. Lassen Sie mich noch vor dem Musterspuk beginnen.

Am Anfang war der Monolith

Hier ist eine Challenge: Schreiben Sie ein Programm, das die Gesamtzahl der Wörter sowie die Zahl der verschiedenen Wörter in einem Text unter Berücksichtigung einer Stopp-

```
internal class Program
{
    public static void Main(string[] args) {
        var stopwords = new string[0];
        if (File.Exists("stopwords.txt"))
            stopwords = File.ReadAllLines("stopwords.txt");

        var text = "";
        if (args.Length > 0)
            text = File.ReadAllText(args[0]);
        else {
            Console.WriteLine("Text eingeben: ");
            text = Console.ReadLine();
            if (string.IsNullOrWhiteSpace(text)) return;
        }

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
            StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;
        var stopwordsDirectory = new HashSet<string>(stopwords);
        var distinctWords = new HashSet<string>();
        foreach (var wortkandidat in candidateWords) {
            foreach (var m in Regex.Matches(wortkandidat, @"[\w-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
        }

        Console.WriteLine($"{countTotal} Wörter, davon {countDistinct} verschieden");
    }
}
```

Eine funktionale Lösung mit kaum vorhandener Struktur (Bild 1)

wortliste bestimmt. Der Text wird entweder vom Benutzer eingegeben oder aus einer Datei gelesen, die der Benutzer beim Programmstart angibt.

Das ist eine simple Aufgabe, denke ich. Dennoch ist alles drin, was eine Software ausmacht: ein bisschen Benutzerschnittstelle, ein bisschen Fachlogik, ein bisschen Datenzugriff. Genug, um darauf das Schichtenmodell und andere Strukturierungsideen anzuwenden. In diesem Beispiel geht es nicht um Technologien. Eine Konsolenanwendung reicht völlig aus. Deren Anwendung könnte so aussehen:

```
$ wordcount.exe
Text eingeben: Es blaut die
Nacht, die Sternlein blinken
```

```
6 Wörter, davon 5 verschieden
$ wordcount.exe gedichtanfang.txt
6 Wörter, davon 5 verschieden
$
```

Der eingegebene Text hat zwar sieben Wörter, doch das Wort „es“ steht in der Stoppwortdatei und wird deshalb nicht gezählt. Außerdem steht „die“ im Text zweimal, daher unterscheidet sich die Zahl der Wörter von der der verschiedenen.

Sie können ja mal als Fingerübung selbst für die Challenge ein Programm schreiben. Beobachten Sie sich dabei: Wie gehen Sie vor? Wie strukturieren Sie den Code und warum?

Wenn Sie mitmachen und später vergleichen möchten, dann lesen Sie erst einmal nicht weiter – Spoileralarm! Denn ich möchte Ihnen verschiedene „hoch entwickelte“ **Lösungen** vorstellen, um daran zu zeigen, warum das **Schichtenmodell und Verwandte keine Option** mehr für mich sind.

Aber zuerst eine Lösungsvariante, wie sie gar nicht mehr geht. Oder wie sie häufig noch anzutreffen ist? Entscheiden Sie, ob Ihnen Code wie in **Bild 1** immer noch begegnet.

Den Code bezeichne ich als **monolithisch**: Er besteht **nur** aus **Logik**. Rekombinierbare Strukturelemente wie Funktio-

```

internal class Program
{
    public static void Main(string[] args) {
        var stopwords = new string[0];
        if (File.Exists("stopwords.txt"))
            stopwords = File.ReadAllLines("stopwords.txt");

        var text = "";
        if (args.Length > 0)
            text = File.ReadAllText(args[0]);
        else {
            Console.WriteLine("Text eingeben: ");
            text = Console.ReadLine();
            if (string.IsNullOrWhiteSpace(text)) return;
        }

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
            StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;
        var stopwordsDirectory = new HashSet<string>(stopwords);
        var distinctWords = new HashSet<string>();
        foreach (var wortkandidat in candidateWords) {
            foreach (var m in Regex.Matches(wortkandidat, @"\w+")) {
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
            }
        }

        Console.WriteLine($"{countTotal} Worte, davon {countDistinct} verschieden");
    }
}

```

Unstrukturierter Code ist ein Flickenteppich aus Verantwortlichkeiten (Bild 2)

nen oder Klassen sind vernachlässigbar. Die Anweisungen in der einzigen Funktion *Main()*, das heißt die Logik, tun zwar das, was sie tun soll – das Programm ist funktional –, doch **verständlich** oder **testbar** ist diese Funktion **nicht**.

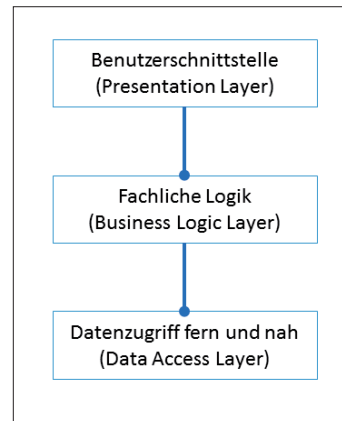
Sicher, das sind kaum 50 Zeilen. Die zu verstehen sollte doch kein Problem sein. Warum hier mehr Aufwand mit mehr Struktur treiben?

Erstens ist das hier ein Beispiel mit überschaubarer Funktionalität, um eben Strukturierungsansätze zu demonstrieren. Selbst wenn das später ein bisschen künstlich und nach Over-engineering aussehen sollte, wird es hoffentlich helfen, die wesentlichen Punkte zu illustrieren, um die es mir geht.

Zweitens glaube ich, dass wir sensibler sein sollten, was die Strukturierung angeht. Wir sollten uns nicht überschätzen in der Fähigkeit, Code zu verstehen. Die **Zeit** für einen Bugfix oder zum Einbau einer Erweiterung ist immer **knapp**. Jede **Minute**, die wir beim Verstehen von Code **sparen** können, bevor wir ihn verändern, ist wichtig. Dafür aber müssen wir vorher, schon beim Schreiben, etwas tun. Der Code-Autor muss an den späteren Code-Leser denken: „**Programs must be written for people to read, and only incidentally for machines to execute.**“ [1]

Was aber macht den Code in **Bild 1** so schwer zu verstehen? Es ist die kunterbunte **Vermischung** von **Verantwortlichkeiten**.

In **Bild 2** habe ich die wesentlichen Verantwortlichkeiten farbig hervorgehoben. Sie sehen, das ist ein Flickenteppich. **Verantwortlichkeiten** sind über die Logik **verstreut**. Sie werden mit Kontrollstrukturen „geöffnet“, um dann andere dazwischenschieben und sie erst später zu „schließen“.



Das Schichtenmodell definiert verständliche Verantwortlichkeiten in klaren Beziehungen (Bild 3)

So kann **kein Fluss des Verständnisses** entstehen. Der Code „**erzählt keine Story**“, in der etwas entlang einer Kausalkette passiert. Es fehlen Bedeutungseinheiten, die Sie auf einen Blick erfassen können. Alles müssen Sie sich durch **Simulation der Ausführung der Anweisungen** erst erschließen.

Das ist gruselig aufwendig und fehlerträchtig. Und ob der Code wirklich korrekt ist, lässt sich nicht ermitteln, ohne ihn manuell auszuführen. **Keine** der **Verantwortlichkeiten** kann gezielt mit **automatisierten Tests** überprüft werden. So lässt sich nicht zügig feststellen, ob der Code schon **reif** zur **Auslieferung** ist oder nach einer Änderung immer noch korrekt, also **regressionsfrei**.

So geht's nicht. Da sind wir uns einig, hoffe ich.

Aber das war die Situation – zumindest früher, auch noch nach Einführung der strukturierten Programmierung. Vor diesem Hintergrund sind die ersten **Architekturmuster entstanden**. Zunächst Model-View-Controller (**MVC**), dann das **Schichtenmodell**. Was für ein Sonnenaufgang über dem Monolithen!

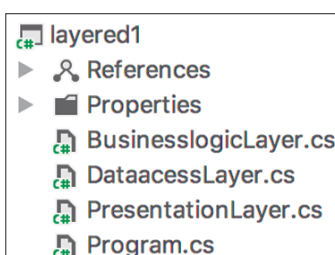
Den Monolithen in Schichten spalten

MVC, Schichtenmodell, Hexagonale Architektur und auch die Clean Architecture verfolgen alle **denselben Ansatz**:

- Sie definieren einen Kanon von Verantwortlichkeiten und verordnen die Spaltung der Logik nach diesen **Verantwortlichkeiten in Module**.
- Sie **geben** genau **vor**, wie die **Module** in **Beziehung** stehen, das heißt, wie sie einander kennen und nutzen dürfen.

Die **Beliebtheit** des **Schichtenmodells** scheint mir hierbei in einer Kombination aus **leicht** zu **verstehen** **den Verantwortlichkeiten** in angenehmer **Granularität** und sehr **klaren Beziehungen** begründet zu sein (**Bild 3**).

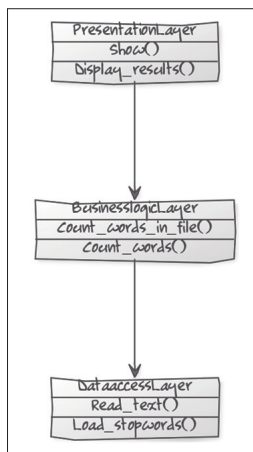
Wenn ich dieses Muster auf den bisherigen Code zur Lösung der obigen Challenge anwende, dann ist damit tatsächlich **etwas gewonnen: Verständlichkeit**. Die **Logik** liegt nicht mehr auf einem Haufen, sondern ist **verteilt** auf **Klassen** als **Module**, sodass sich schon beim Betrachten des Projekts eine gewisse Übersicht einstellt (**Bild 4**). ►



Schichten steigern die Übersichtlichkeit (Bild 4)

Wer mit dem Schichtenmodell vertraut ist, erkennt hier Verantwortlichkeiten, unter denen er sich **etwas vorstellen kann**. Zudem sind dem Betrachter sofort die **grundsätzlichen Beziehungen klar**. Das ist auch der Sinn der Einhaltung eines solchen Musters.

Bild 5 zeigt konkret die **Schichtung der Klassen** der Umsetzung aus Bild 4. **Ordentlich**, oder? Eine so organisierte Codebasis macht Freude. Alles hat seinen Platz. Da wissen Sie genau, wo was passiert. Ja, damit ist etwas gewonnen. Es ist **besser als vorher**, aber noch nicht wirklich gut. Denn schauen Sie sich einmal den Code der Businesslogik an (Bild 6).



Konkrete Schichtung der Anwendungslogik (Bild 5)

```

class BusinesslogicLayer
{
    public static (int countTotal, int countDistinct) Count_words_in_file(string filename) {
        var text = DataaccessLayer.Read_text(filename);
        return Count_words(text);
    }

    public static (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = DataaccessLayer.Load_stopwords();
        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
            StringSplitOptions.RemoveEmptyEntries);
        var countTotal = 0;
        var countDistinct = 0;
        var distinctWords = new HashSet<string>();
        foreach (var wordCandidate in candidateWords) {
            foreach (var m in Regex.Matches(wordCandidate, @"(\w+)-")) {
                if (!string.IsNullOrEmpty(m.ToString())) {
                    var word = m.ToString();
                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;
                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
            }
        }
        return (countTotal, countDistinct);
    }
}
    
```

Schlecht testbare Logik trotz Schichtung (Bild 6)

Die Verantwortlichkeiten sind sauber getrennt, die Abhängigkeiten ausgerichtet – doch gut testbar ist deshalb die eigentliche Businesslogik immer noch nicht. Denn die **Businesslogik hängt** immer noch **von der Datenzugriffslogik ab**. Es besteht eine **funktionale Abhängigkeit**: **Logik** in einer **Methode ruft** eine **andere Methode auf**, um zwischendurch deren Logik auszuführen.

Das hört sich normal an und findet sich bestimmt in Ihrem Code auch allerorten. Doch das macht es nicht besser. Solche **funktional abhängige Logik** ist schlicht **nicht für sich allein testbar**.

Natürlich ist die Logik in Bild 6 auch in anderer Hinsicht noch suboptimal. Doch das ist sekundär für den Punkt, um den es mir hier im Moment geht. Ich habe nur das minimal Nötige getan, um den monolithischen Code nach dem Schichtenmodell zu strukturieren. Das **fundamentale Problem** des **Schichtenmodells** geht **nicht weg**, wenn ich die Wortzählungslogik noch weiter refaktoriere. Der Klumpen in `Count_words()` dient also der Unterstreichung des grundsätzlich zu lösenden Problems der funktionalen Abhängigkeiten.

Wer die **Businesslogik testen** will, kann das im Moment trotz – oder wegen – Schichtenmodell **nur** tun, indem er ebenfalls die **Logik** der **Datenzugriffsschicht ausführt**. Das macht einen **Businesslogik-Test langsamer** und/oder **umständlicher**, weil eine Datei als Ressource bereitgestellt werden muss.

Nicht wirklich dramatisch in diesem trivialen Beispiel, doch wenn Sie sich das Szenario umfangreicher denken, kommt schon einiges an Overhead zusammen.

Wenn Bild 1 den Bewusstseinsstand in Sachen Anwendungsarchitektur bis **Mitte** der **1990er-Jahre** in vielen Projekten widerspiegelt, dann steht Bild 6 für **Ende** der **1990er**.

Schichten entkoppeln

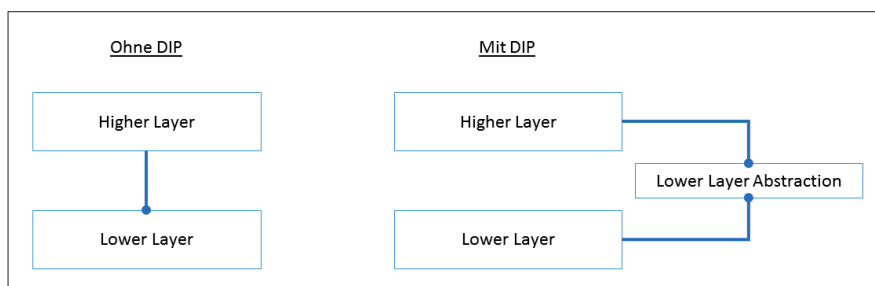
In einer Co-Evolution mit bewussterer Anwendungsstrukturierung befand sich ab **Ende** der **1990er-Jahre** das Thema **Testen**. Die ersten Unit-Testing-Frameworks kamen auf.

Wo klare Verantwortlichkeiten in Modulen freigestellt waren, konnten überhaupt erst automatisierte Tests feingranular ansetzen. Aber um in einer sauberen Hierarchie automatisierte Tests punktgenau nur gewisse **Logik testen** lassen zu können, **brauchte** es **Entkopplung der Verantwortlichkeiten**.

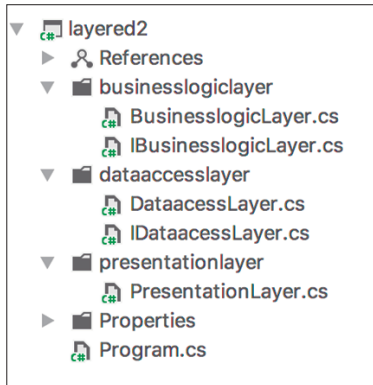
Auftritt **DIP**: Mit dem Dependency Inversion Principle wurde es möglich, Tests auf eine Schicht zu fokussieren.

Der **Trick** besteht darin, **Compilezeitabhängigkeiten** von **Laufzeitabhängigkeiten** zu **trennen**. Zur **Compilezeit** besteht nach **DIP keine direkte funktionale Abhängigkeit** von Logik einer **Schicht** zu Logik einer **anderen**. Eine obere Schicht hängt nicht von einer konkreten unteren ab, sondern **lediglich** von einer **Abstraktion** (Bild 7).

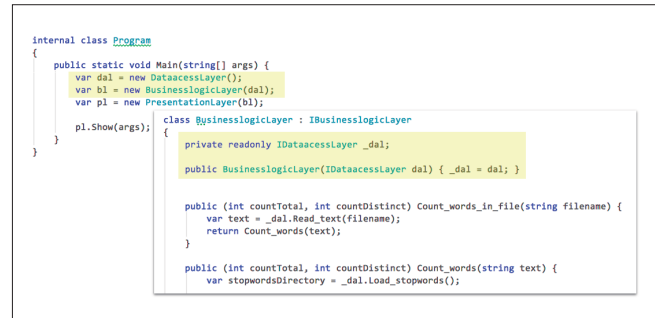
Abstraktionen sind gewöhnlich Interfaces. Die können von der niedrigen Schicht implementiert werden – aber man kann auch eine **Attrappe** für eine niedrige Schicht so aussehen lassen. Doch eins nach dem anderen. Zuerst zeigt Bild 8 die Anwendung mit **verbesserter Schichtenarchitektur** im Überblick. Der geübte Softwerker lässt beim Anblick dieser Module sogleich vor seinem geistigen Auge ein Beziehungsgeflecht wie in Bild 9 entstehen und weiß: Alles **hübsch entkoppelt** und **testbar**. In Bild 10 sehen Sie, wie mit einer Attrappe die Businesslogik unabhängig von darunterliegenden Details getestet wird. Statt einen Datenzugriff konkret zu durchlaufen, werden die Stopwörter im Test hart verdrahtet. Das ist trivial in puncto Laufzeit und Komplexität und **einfacher, als** eine **Stopwortdatei** zu benutzen.



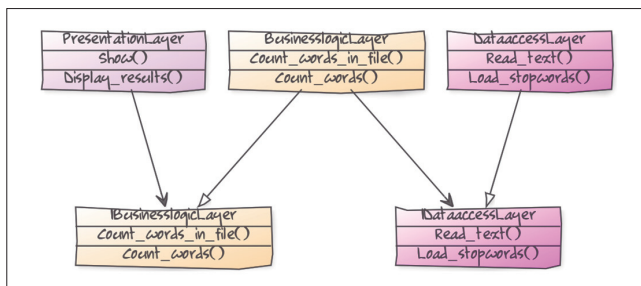
Mit dem DIP werden Schichten entkoppelt (Bild 7)



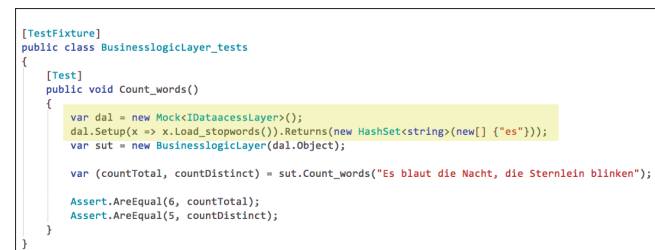
Eine Schichtenarchitektur mit DIP (Bild 8)



Injizieren der konkreten Implementation einer abstrakten Abhängigkeit zur Laufzeit (Bild 11)



Über Interfaces entkoppelte Schichten (Bild 9)



Mit einer Attrappe wird das Testen von abhängiger Logik einfach (Bild 10)

Dass der Businesslogik überhaupt eine Attrappe untergeschoben werden kann, liegt an der **Anwendung** des Prinzips **Inversion of Control (IoC)**. Dessen Manifestation besteht hier im Hineinreichen der **Laufzeitabhängigkeit** in die Businesslogik durch ihren Konstruktor (Constructor **Injection**, Bild 11).

Die **Abhängigkeit** vom **Interface IDataaccess** zur **Compilezeit** wird zur **Laufzeit** durch die **Injektion** einer **Implementation** des Interfaces befriedigt. In Bild 12 sind die Compilezeit- und Laufzeitabhängigkeiten zusammen visualisiert. Das sieht jetzt schon **nicht mehr so einfach** aus wie das ursprüngliche Schichtenmodell, würde ich sagen. **Logik** ist auch im Schichtenmodell bei vortrefflicher Ausrichtung der Beziehungen **immer noch funktional abhängig**. Um trotz dieser Abhängigkeiten Testbarkeit zu erlangen, müssen **zusätzliche Elemente** eingeführt werden: **Abstraktionen** (hier: Interfaces). Und aus einer Menge unidirektionaler Abhängigkeiten werden zwei Mengen, von denen eine auch noch gegenläufige Abhängigkeiten enthält.

Das scheint der Preis der Wandelbarkeit zu sein. **Verständlichkeit** entsteht **durch Trennung** von **Verantwortlichkeiten** und **klare Beziehungen**. **Testbarkeit** entsteht **durch DIP und IoC**. Ist eben so. Da müssen wir durch. Um das Leben aber nun wenigstens ein bisschen einfacher zu machen, gibt es Mock-Frameworks wie Moq (wie in Bild 10 benutzt) und Dependency-Injection-Frameworks wie MEF oder Unity.

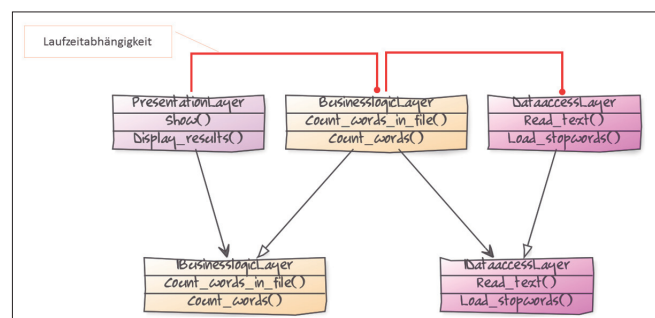
Jetzt ist es **nur noch eine Sache konsequenter Anwendung von Prinzipien und Werkzeugen**, um sauberen Code zu schreiben. Zur grundlegenden Strukturierung von Logik scheint damit alles gesagt. Das ist zumindest der Stand des Bewusstseins, den ich bei Clean-Code-Development-Trainings der CCD School oft sehe. Wenn ein Architekturmuster

bekannt ist, dann ist es MVC oder das Schichtenmodell. Zusätzlich wird dann noch die Fahne der SOLID-Prinzipien hochgehalten, zu denen das DIP gehört, sowie das SRP (Single Responsibility Principle), das sich mit Verantwortlichkeitstrennung befasst. Nur leider sehe ich **auch bei den Teams**, die Code **SOLID-gemäß** in **Schichten strukturieren**, **keine entspannten oder freudvollen Gesichter**. Der Code ist **immer noch schwer** zu **wandeln**. Sonst würde man mit mir ja auch nicht über Clean Code Development sprechen wollen.

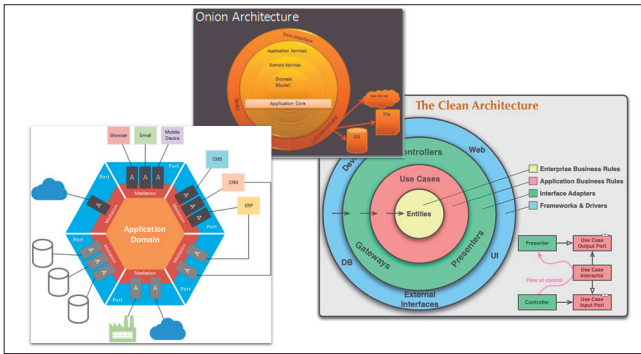
Wie kann das aber sein? **Trotz sauberer Schichtung immer noch nicht sauber?** Merkwürdig, oder?

Schichten in Schale werfen

Dass daran irgendetwas faul ist, ist denn auch der Branche schon bewusst geworden. Seit **2005** hat sich das **Schichtenmodell gewandelt**. Aus den Schichten sind Schalen der einen oder anderen Art geworden. Statt aus Kästchen bestehen ►



Mit DIP unterscheiden sich die Abhängigkeiten zur Compilezeit und zur Laufzeit (Bild 12)



Konzentrische Architekturmuster (Bild 13)

die empfohlenen Architekturmuster nun aus konzentrischen Kreisen. Verantwortlichkeiten sind **ineinander** organisiert **statt übereinander**. Vertreter solch konzentrischer Architekturmuster sind die **Hexagonale** Architektur, die **Onion** Architecture und die **Clean** Architecture (Bild 13).

Treiber für diese **neue Grundorganisation** ist eine Bedingungsaufladung der Verantwortlichkeiten. Dass es dezidierte Verantwortlichkeiten gibt und dass es eine klare Richtung für Abhängigkeiten gibt, hat sich nicht verändert. In der konzentrischen Anordnung drückt sich vielmehr aus, dass man erkannt hat, dass **nicht alle Verantwortlichkeiten gleich** sind. Irgendetwas ist **anders** mit dem, was **im Kern** steht, im Vergleich **zu** dem, was am **Rand** steht.

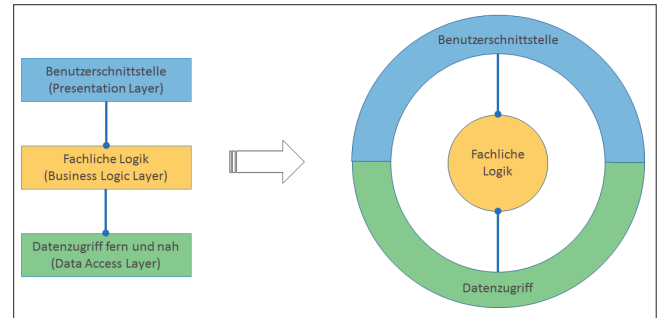
Ich hatte mich auch schon lange beim Schichtenmodell gefragt, **warum Businesslogik** überhaupt **von Datenzugriffslogik abhängig** sein soll. Warum sollte sich eine Geschäftsregel damit befassen, Daten zu beschaffen oder zu speichern? Dass die Businesslogik Daten braucht, auf denen sie arbeitet, und Daten erzeugt, ist klar. Muss sie dafür aber den **Datenzugriff selbst steuern**? Mir will das nicht in den Kopf. Das sieht für mich wie eine **Vermischung von Verantwortlichkeiten** aus – auch wenn es irgendwie so ganz normal erscheint.

In den konzentrischen Architekturen ist dieses Problem gelöst – zumindest scheinbar. Denn in **konzentrischen Architekturen** weisen die **Abhängigkeiten von außen nach innen auf die Businesslogik hin** (Bild 14). Die **Businesslogik steuert** nun augenscheinlich **nicht** mehr den **Datenzugriff**.

Das ergibt mehr Sinn, finden Sie nicht auch? **Je weiter außen** eine Schale, desto mehr beschäftigt sie sich mit **technischen Details** und desto **volatiler** ist sie. **Je weiter im Inneren** eine **Verantwortlichkeit**, desto **zentraler** ist sie, im wahrsten Sinne des Wortes, desto **zeitloser** und **konstanter**.

In der **Clean Architecture** (Bild 13) stehen **Entities** für sehr **allgemeine**, grundlegende **Regeln**. „Enterprise Business Rules“ können sich naturgemäß **nicht** so **häufig** und **radikal ändern** wie „Application Business Rules“ der darum herum liegenden Use Cases.

Die **konzentrischen Architekturmuster adressieren** also den **Aspekt der Volatilität** zum



Konzentrische Abhängigkeiten ergeben mehr Sinn (Bild 14)

Bild. Von **außen nach innen**, also in **Richtung der Abhängigkeiten**, wird **Code immer „ruhiger“**. Robert C. Martin sagt in seinem **Buch Clean Architecture** [2]: „The outer circles are mechanisms. The inner circles are policies.“ „Mechanisms“, also **technische Vorrichtungen**, sind demnach **eher Änderungen unterworfen als „Policies“** (Regeln). Für Martin definieren die Schalen Ebenen: „[T]he further in you go, the higher level the software becomes.“ Aber was für Ebenen? Was ist das Kriterium, um eine Ebene zu bestimmen?

„As you move inward, the level of abstraction and policy increases.“ Die Ebene, auf der Regeln liegen, wächst nach innen. Und auch das Abstraktionsniveau wächst nach innen. Jedenfalls für Robert C. Martin.

Bei den **Regeln gehe ich noch mit**. Beim **Abstraktionsniveau** bin ich jedoch **anderer Meinung**. Dazu **später mehr**.

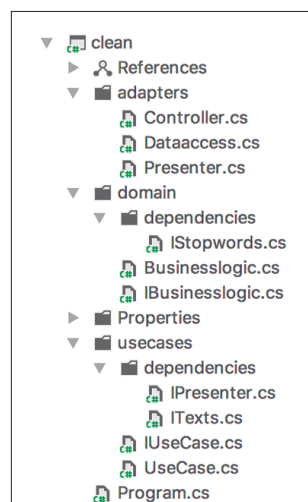
In jedem Fall erweitern die konzentrischen Architekturmuster das Bild um eine Dimension. Das ist gut so. Verantwortlichkeiten liegen nicht alle auf einer Ebene, egal was dafür das Kriterium ist. Was bedeutet das jetzt für den Code?

Bild 15 zeigt meine Bemühung, das Muster der Clean Architecture auf das bisherige Beispiel zu übertragen. Bemüht habe ich mich, weil ich es Ihnen einerseits leicht machen möchte, zu sehen, wohin die Verantwortlichkeiten aus **Bild 8** gewandert sind; meine Clean Architecture soll also vergleich-

bar sein. Andererseits wollte ich aber auch nicht zu weit von den Begriffen bei Robert C. Martin abweichen; die Umsetzung soll also auch mit dem Muster vergleichbar sein.

Am einfachsten ist es für Sie bestimmt, die **Schalen** der Clean Architecture in **Bild 15** zu identifizieren. Von **außen nach innen** stehen dafür die Projektverzeichnisse **adapters**, **usecases**, **domain**. Als **Adapter** verstehe ich die Klassen, die den **Zugriff auf Hardware** kapseln, zum Beispiel **Tastatur** oder **Festplatte**. Das sind Martins „Mechanisms“.

Im **Kern** steht für mich die **Domäne**, bei der ich nicht weiter unterscheide, ob es sich um **Entities** oder **Services** handelt (wenn ich einmal das Vokabular aus dem Domain Driven Design (DDD) in Anschlag bringe). Die Verantwortlichkeiten **Dataaccess** und **Businesslogic** haben sich auch **nicht** sehr **verändert**.



Code-Artefakte in einer Clean Architecture (Bild 15)

```

class UseCase : IUseCase
{
    private readonly IBusinesslogic _bl;
    private readonly ITexts _txt;
    private readonly IPresenter _ui;

    public UseCase(IBusinesslogic bl, ITexts txt, IPresenter ui) {
        _bl = bl;
        _txt = txt;
        _ui = ui;
    }

    public void Count_words_in_file(string filename) {
        var text = _txt.Retrieve(filename);
        Count_words(text);
    }

    public void Count_words(string text) {
        var (countTotal, countDistinct) = _bl.Count_words(text);
        _ui.Display_results(countTotal, countDistinct);
    }
}

```

Der Use Case beim Orchestrieren von Adaptern und Businesslogik (Bild 16)

Allerdings ist der vormalige *PresentationlogicLayer* nun zerfallen in *Controller* und *Presenter*.

Die Benutzerschnittstelle besteht aus zwei Teilen, weil der Use Case neu hinzugekommen ist. Dieser ist in der Clean Architecture ganz explizit und hat laut Martin folgenden Auftrag: „These use cases orchestrate the flow of data to and from the entities [...]“ Dafür ist im vorliegenden Fall nicht viel zu tun, aber ich habe mich deshalb zum Beispiel entschlossen, aus der Businesslogik das Laden des Textes herauszunehmen (Bild 16). Außerdem: Der Use Case „schiebt“ das Ergebnis der Wortzählung nun an einen Presenter weiter, statt es seinem Aufrufer – dem Controller – zurückzugeben. Das ist in der Clean Architecture auch ganz explizit beschrieben (Bild 17).

Tatsächlich trägt das „Anhängsel“ der Clean Architecture in Bild 17 wesentlich dazu bei, den Code gemäß der Forderung „Alle Compilezeitabhängigkeiten weisen nach innen“ zu strukturieren. Ihm ist es geschuldet, dass die Implementation in Bild 15 fünf Interfaces statt bisher drei hat und diese auch noch auf zwei Ebenen definiert sind. Beispiel Schale *domain*: Die umschließende Schale *usecases* kennt von der Domäne nur das Interface *IBusinesslogic*, so wie der bisherige *PresentationlogicLayer* von der darunterliegenden Schicht auch nur dieses Interface kannte. So weit, so einfach und naheliegend.

Zusätzlich jedoch definiert die Schale *domain* das Interface *IStopwords*. Das zählt für mich zu einer anderen Kategorie, daher habe ich es in das Verzeichnis *dependencies* gesteckt. *IStopwords* wird nicht von der Businesslogik implementiert,

```

class Businesslogic : IBusinesslogic
{
    private readonly IStopwords _dal;

    public Businesslogic(IStopwords dal) { _dal = dal; }

    public (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = _dal.Retrieve();

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
                                         StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;
    }
}

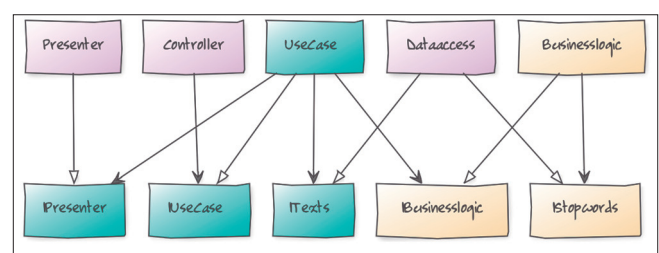
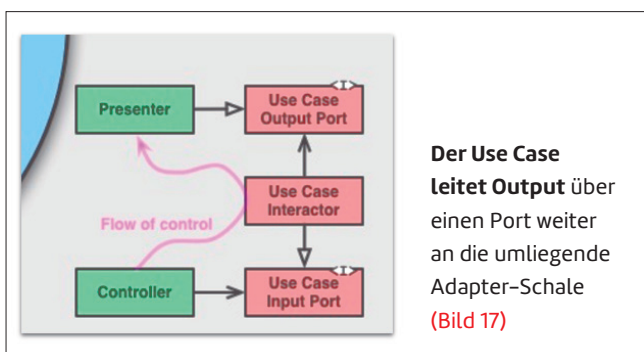
```

Zur Laufzeit ruft eine innere Schale eine äußere auf (Bild 18)

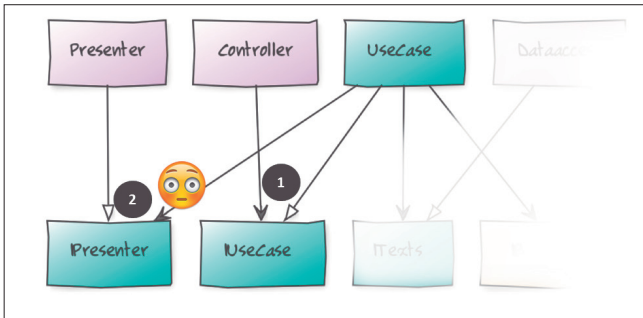
sondern von einer weiter außen liegenden Schale! Genauer: von der Klasse *Dataaccess* in *adapters*. *IStopwords* ist in *domain* definiert, um der Forderung nach den Verweisen nach innen zu genügen. Die äußere *adapters*-Schale hängt nun ganz sauber von der inneren *domain*-Schale ab – obwohl immer noch Businesslogik sich die Stoppwörter selbst beschafft (Bild 18). Das Schichtenmodell war da aus meiner Sicht ehrlicher. Dort verliefen die grundsätzlichen Abhängigkeiten von oben nach unten, so wie auch zur Laufzeit der Kontrollfluss. In der Clean Architecture ist das nicht mehr der Fall!

Vollziehen Sie das wirklich einmal anhand von Bild 15 und Bild 18 nach. Lassen Sie es sich auf der Zunge zergehen. Nehmen Sie Bild 17 zu Hilfe. Hier geht um den Kern der konzentrischen Architekturen! Das ist so wichtig, dass ich es Ihnen aufmale. Zunächst die Compilezeitabhängigkeiten in Bild 19. Blicken Sie hier durch? Das ist alles wunderbar nach DIP entkoppelt. Der Use Case ist nicht von einer Domänenimplementation abhängig (Pfeil mit gefüllter Spitze), sondern nur von einer Abstraktion. Ebenfalls von dieser abhängig ist ihre Implementation (Pfeil mit offener Spitze), die Businesslogik. Ebenso ist es mit den anderen Implementationen, die in Bild 19 alle in der oberen Zeile stehen: Keine Pfeile zwischen ihnen. Alle Pfeile weisen nach unten auf Abstraktionen.

Aber nicht nur die größere Zahl an Interfaces macht das Bild hier verwirrender. Hinzu kommt noch die Zuordnung von Abstraktionen und Implementationen zu Schalen. Schauen Sie genau, wie die Beziehung zwischen äußeren und inneren Schalen wechselt. In Bild 20 zoomte ich beispielhaft heran. Der Controller in der äußeren Schale kennt nur *IUseCase* als Abstraktion der inneren Use-Case-Schale (1). Die Abstraktion gehört zur inneren Schale, die sie auch implementiert. Gleichzeitig (!) ist die innere Schale von der Abstraktion *IPresenter* abhängig, die jedoch von der äußeren implementiert wird (2). Verwirrend, oder? Jedenfalls empfinde ich das so. ►



Die Abhängigkeiten gemäß DIP in der Clean Architecture (Bild 19)



Zur Compilezeit hängt die äußere von der inneren Schale ab (Bild 20)

Und Robert C. Martin gibt in seinem Buch für ein ähnliches Beispiel zu: „Much of the complexity in that diagram was intended to make sure that the dependencies between the components pointed in the correct direction.“

Bitte lesen Sie das Zitat noch einmal! Komplexität ist der Feind einfacher Wandelbarkeit. Clean-Code-Prinzipien und -Praktiken sollten sich also darum bemühen, Code-Komplexität zu reduzieren. Und als Beispiel für Sauberkeit gilt dann ein Architekturmuster, über das der Pate der Clean-Code-Bewegung sagt, es sei komplex, damit Abhängigkeiten in eine gewisse Richtung weisen? Komplexität für Sauberkeit?

Ich muss sagen, dass ich nicht bereit bin, das zu akzeptieren. Diesen Preis möchte ich nicht bezahlen. Wandelbarkeit in Form von Verständlichkeit und Testbarkeit möchte ich nicht dadurch bekommen, dass ich Komplexität bewusst in diesem Ausmaß erhöhe. Ich habe es bei der Implementation des Beispiels als Clean Architecture schon bemerkt, und nun, als ich Bild 19 erarbeitet habe, ist es mir noch deutlicher geworden: Was in konzentrischen Kreisen noch klar aussieht und eine simple Regel definiert – „Abhängigkeiten weisen immer nach innen“ –, verliert diese Eigenschaften im Code.

Die Krone wird dem aufgesetzt, wenn Sie sich klarmachen, wie die Laufzeitabhängigkeiten aussehen (Bild 21). Die weisen nämlich von außen nach innen und dann wieder nach außen. Das gute alte Schichtenmodell ist zurück. Wofür also die Steigerung der Komplexität? Was ist wirklich gewonnen durch die konzentrischen Architekturmuster?

Reflexion

Früher war nicht alles besser. Codezustände wie in Bild 1 will niemand (wieder) haben.

Die erste Variante des Schichtenmodells jedoch, die war gar nicht so schlecht. Die klare Trennung von Verantwortlichkeiten, kombiniert mit einer konsequenten Ausrichtung der Abhängigkeiten, hat die Verständlichkeit deutlich gesteigert (Bild 4 und Bild 5).

Für gute Wandelbarkeit war das allerdings noch nicht genug. Die braucht nicht nur Verständlichkeit, sondern auch Testbarkeit. Diese war in der ersten Schichtenvariante noch begrenzt. Lediglich die Logik der untersten

Schicht war gut testbar, weil sie für sich stand. Die Logik in den darüberliegenden Schichten konnte zwar grundsätzlich schon gezielt angesprochen werden, nur müsste dann auch immer die Logik darunterliegender Schichten beim Test mit durchlaufen werden. Das kostet Zeit, und das macht es nicht leicht, einen Bug zu lokalisieren.

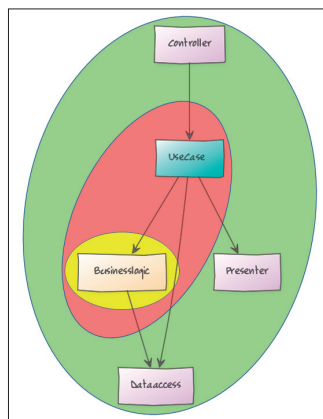
Die Testbarkeit ist dann in der zweiten Variante des Schichtenmodells (Bild 8 und Bild 9) nachgezogen worden. Durch Anwendung von DIP und IoC können für Tests untere Schichten ausgeblendet werden. Wenn etwas schiefgeht, dann weiß man, dass der Fehler in der Logik der zu testenden Schicht steckt. Allerdings: Dieser Fortschritt in der Testbarkeit hat seinen Preis. Der wird deutlich in der Clean-Architecture-Variante (Bild 15 und Bild 19). DIP und IoC addieren Komplexität, die die Verständlichkeit nun – zumindest nach meinem Empfinden – massiv reduziert. Man ist über das Ziel hinausgeschossen. „Mehr vom selben“ (hier: DIP und IoC) hat den Fortschritt, den die zweite Variante des Schichtenmodells gebracht hat, nicht vergrößert. Im Gegenteil!

Aber wie kommt das? Ich glaube, es liegt daran, dass man zu sehr auf die Compilezeitabhängigkeiten gestarrt hat.

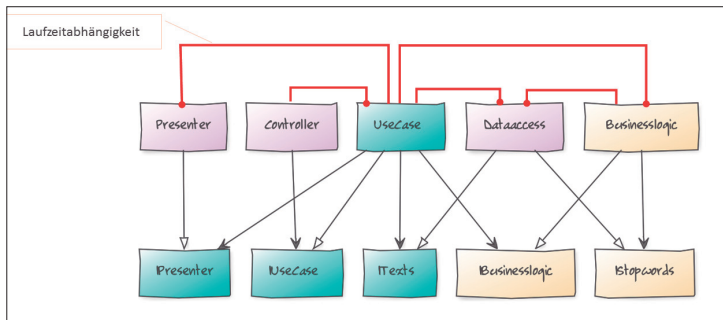
Von der Schichtung zur Konzentrik überzugehen hat etwas mit den Compilezeitabhängigkeiten zu tun. Im Schichtenmodell war das Volatile (oder Instabile) – Robert C. Martins „Mechanisms“ – nicht konsequent in einer Position, wo Veränderungen wenig Probleme machen. Relativ problemlos sind Veränderungen nämlich dort, wovon nur wenige oder keine Codeeinheiten abhängig sind. Im Schichtenmodell ist das nur für die Präsentationslogik der Fall. Die Datenzugriffslogik hingegen, die ebenfalls ein „Mechanism“ ist, muss – wegen der Abhängigkeit anderer von ihr – Stabilität zusichern. Das wurde mit der Clean Architecture bewusst verändert. Dort sind nicht nur die Abhängigkeiten sauber ausgerichtet, sondern auch die Verantwortlichkeiten nach Stabilität positioniert: Am stabilsten sind ganz allgemeine, grundlegende Regeln im Kern, am instabilsten die Kommunikation mit der Umwelt in der äußeren Schale.

Diese Sichtweise gefällt mir – allerdings hat die Implementation eben einen hohen Preis. Bild 17 und Bild 20 machen es exemplarisch deutlich; die Verständlichkeit der Zusammenhänge im Code sinkt.

Aber selbst wenn ich einmal über den auch von Robert C. Martin beklagten Mehraufwand hinwegsehe, frage ich mich, was wirklich gewonnen ist. Denn Bild 21 zeigt ein Bild, das sich im Grunde nicht vom Schichtenmodell unterscheidet. Zur Laufzeit ist eine Businesslogik immer noch vom Datenzugriff abhängig. Was soll das? Da mag zur Compilezeit die Abhängigkeit umgekehrt sein – Adapter außen hängt von Domäne innen ab –, doch zur Laufzeit gehen die Aufrufe dorthin durch die Domäne in die Tiefe. Es ist letztlich nichts gewonnen. Die Abstraktion, von der die Businesslogik abhängig ist, ist lediglich verschoben worden. Vorher ge-



Die Laufzeitabhängigkeiten zeigen in der Clean Architecture von außen nach innen und wieder nach außen (Bild 21)



Das vollständige Abhängigkeitsbild der implementierten Clean Architecture (Bild 22)

hörte sie zur darunterliegenden Schicht (*IDataaccessLayer* in Bild 9), jetzt gehört sie zur Businesslogik selbst (siehe *IStopwords* in Bild 19). Dass nichts gewonnen ist, ist beim Testen deutlich zu bemerken. Das Architekturbild suggeriert, dass eine innere Schale keine Abhängigkeit zu einer äußeren hat – doch beim Testen stellt sich das Gegenteil heraus. In Bild 17 ist der *Use Case Interactor* – also Code der Use-Case-Schale – zur Laufzeit abhängig vom *Presenter* in der darüberliegenden Schale. Dass der Use Case Output Port zur Use-Case-Schale gehört, kaschiert das nur. Im Test muss trotzdem eine Attrappe gebaut werden.

Sind Sie noch da oder haben Sie schon halb abgeschaltet? Das würde mich nicht wundern. Bei dem ganzen Hin und Her der Abhängigkeiten von Compilezeit und Laufzeit kann einem schon der Kopf schwirren. Ich jedenfalls verliere bei der Darstellung der vollständigen Abhängigkeiten der implementierten Clean Architecture den Überblick (Bild 22).

So sieht für mich ein Testalbtraum aus. Ganz zu schweigen davon, dass damit das Rätselraten darum, für welche Klassen ein Interface definiert werden sollte, weiter angeheizt wird. Im Zweifelsfall lautet dann die Antwort „Für alle!“, und damit explodiert die Zahl der Dateien in einem Projekt (wenn

man je Klasse und je Interface eine Datei denkt). Das einzig Positive, das ich der Clean Architecture abgesehen vom hübschen Bild abgewinnen kann, ist die Tendenz zur Aufspaltung von Abstraktionen. Sie scheint die Anwendung des Interface Segregation Principle (ISP) nahezulegen. Schmalere Interfaces helfen einfach bei der Entkopplung.

Wo vorher nur eine Präsentationslogikschicht und eine Datenzugriffsschicht mit entsprechenden Abstraktionen waren, sind beide nun in mehrere Teile zerfallen: Controller und Presenter sind getrennt und es gibt *IPresenter*; ebenfalls getrennt sind *IText* und *IStopwords*, wo vorher nur *IBusinesslogicLayer* war.

Am Ende ist dieser positive Effekt für mich jedoch nicht ausschlaggebend. Viel wichtiger finde ich das Sinken der Verständlichkeit durch eine gestiegene Artefaktzahl und den weiterhin hohen Testaufwand. Denn jede Laufzeitabhängigkeit ruft zur Testzeit nach einer Attrappe.

- [1] Harold Abelson & Gerald Jay Sussman, *Structure and Interpretation of Computer Programs*, MIT Press, 2. Auflage, 1996, ISBN 978-0-262-51087-5
- [2] Robert C. Martin, *Clean Architecture, A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017, ISBN 978-0-1344-9416-6



Ralf Westphal

ist freiberuflicher Berater, Referent, Autor und Trainer für Themen rund um Softwarearchitektur und die Organisation von Softwareteams. Er ist Mitgründer von Clean Code Developer (CCD) und propagiert kontinuierliches Lernen.

info@ralfw.de

dnpcode

A1808IODA

Wir suchen zum nächstmöglichen Zeitpunkt
am Standort Nürnberg einen

Web-Entwickler (m/w) ASP.NET, C# und JavaScript

Ihr Profil

Abgeschlossenes Universitätsstudium der Fachrichtung Informatik

Erfahrung in der Entwicklung von Webanwendungen mit ASP.NET und JavaScript

Fundierte Kenntnisse in der Oberflächen-gestaltung mit HTML und CSS

Ihre Aufgaben

Entwicklung unserer Software-Suite DeltaMaster unter Einsatz moderner Webtechnologien in agilen Entwicklungsprozessen einschließlich Testing

Jetzt bewerben unter jobs@bissantz.de

www.bissantz.de/jobs

BISSANTZ

